

Algoritmos y Estructuras de Datos
Trabajo Práctico 3 – Enunciado Base 2026
Implementación de un Sistema Simple de Gestión de Tratamientos Médicos v3.0

- **Aclaración inicial**

La comprensión de este texto en su totalidad es parte de lo que se evalúa, esto implica que no respetar los formatos esperados para las entradas y salidas o los resultados esperados será penalizado con baja de puntos o incluso aplazo. En función de esto, se les recomienda encarecidamente leer TODO el archivo antes de sentarse a desarrollar una solución. En caso de que esta primera lectura no sea esclarecedora, se recomienda volver a leer hasta tanto cada uno de los elementos del texto sean comprendidos y tenidos en cuenta en su totalidad.

- **Introducción:**

Se pedirá desarrollar una tercera versión (la versión 3.0) del mismo sistema simple que se planteó para el TP1 y el TP2 (gestión de tratamientos médicos en un centro de salud) en base al mismo esquema de identificación de las enfermedades, problemas de salud y causas de muerte del modelo ICD10 (*International Classification of Diseases, versión 10*) ya usado en el TP1 y en el TP2 (vea el enunciado del TP1 y del TP2 para recordar los detalles, que deberá volver a tener en cuenta en este nuevo trabajo).

- **Entradas:**

En esta versión deberán procesar varios tratamientos, y los datos de estos tratamientos vendrán en un archivo de texto con formato CSV (valores separados por comas) que contiene una línea por cada tratamiento a procesar. La primera línea del archivo, es una línea especial que indica los nombres de los campos que contiene cada tratamiento y debe ser ignorada.

Cada línea del archivo contiene los siguientes campos (en el orden que se detalla a continuación):

1. **DNI del Paciente (sin puntos):** Número entero representando el DNI del paciente
2. **Nombre del paciente:** Cadena de caracteres representando el nombre del paciente
3. **Apellido del paciente:** Cadena de caracteres representando el apellido del paciente
4. **Código ICD10 que identifica el diagnóstico:** Cadena de caracteres con el código ICD10 de acuerdo a las mismas especificaciones que en los TPs anteriores
5. **Monto base a pagar por el tratamiento:** Número flotante que representa el monto base/mínimo a pagar por el tratamiento
6. **Si el tratamiento es de alta complejidad o no:** Un caracter que representa si es de alta complejidad o no. El caracter "A" representa un tratamiento de alta complejidad, el caracter "R" representa un tratamiento regular.
7. **Identificador del algoritmo de cálculo de monto final:** Número entero que representa el identificador (o ID) del algoritmo que se debe utilizar para calcular el monto final. Los algoritmos están especificados más adelante en este documento, pero todos los algoritmos en la lista tienen las mismas entradas y salidas:
 - a. **Entradas:** Código ICD10, tipo de tratamiento (alta complejidad o no) y monto base a pagar.
 - b. **Salidas:** Monto final a pagar.

El programa será ejecutado y evaluado mediante un programa de control (o runner), y ese runner espera que los datos se carguen desde el archivo de texto **exclusivamente**. Por lo tanto, **insistimos fuertemente en eso: el programa que tienen que realizar no debe tomar datos desde el teclado (salvo para el ingreso de la opción seleccionada del menú)**. Tampoco debe haber en ninguna parte **nada** que solicite al usuario presionar una tecla para continuar. Todos los datos que el programa debe procesar (y solo esos datos) estarán en el archivo de texto **tratamientos.csv** cuyo formato y contenido se describe más abajo.

Es MUY IMPORTANTE que ese archivo se llame **tratamientos.csv** y que esté almacenado en la misma carpeta del proyecto que contiene al programa fuente que desarrollen. Si están haciendo pruebas con otros archivos, no hay problemas, pero ASEGÚRENSE de que cuando entreguen el programa para su calificación, la función `open()` esté abriendo exclusivamente un archivo llamado **tratamientos.csv** Y NO OTRO.

Ejemplo: En el siguiente modelo mostramos qué forma podría tener un pequeño archivo de texto como el indicado:

tratamientos: Bloc de notas	
Archivo	Edición Formato Ver Ayuda
DNI del Paciente,Nombre del paciente,Apellido del paciente,Codigo ICD10,Monto base,Complejidad,ID_Algoritmo	
31947812,Jorge,Gimenez,883.5,37212.07,R,2	
28139536,Camila,Rojas,I66.0,51466.09,A,13	
24048678,Nicolas,Torres,I10.6,31126.44,R,8	

La tabla que sigue muestra los algoritmos que deben ser aplicados para el cálculo del monto final de cada tratamiento (de acuerdo al campo 7 del archivo csv de entrada). Si el código de algoritmo informado para un tratamiento en el archivo csv NO ESTÁ en esta tabla, entonces el monto final para ese tratamiento debe ser calculado en forma normal, de acuerdo a lo indicado en el TP2.

Tabla de algoritmos de cálculo del monto final		
ID	Detalle	Aclaraciones
1	Si el monto_base es menor o igual a 60000, entonces porcentaje_extra = 0. Si el monto_base es mayor a 60000, entonces: - Calcular porcentaje_extra en forma normal. - Si el tratamiento es de alta complejidad y el ICD10 no es "U" agregar una suma fija igual a la mitad del monto base original. Sea monto_final = monto_base + porcentaje_extra + suma_fija.	Si el tratamiento NO es de alta complejidad, la suma fija debe quedar en 0. Lo mismo vale si es de alta complejidad pero el ICD10 es "U".
2	Si la letra del ICD10 está entre "A" y "P" (ambas incluidas), entonces porcentaje_extra = cálculo normal sin importar la complejidad del tratamiento. Para todo otro ICD10: Si el tratamiento es de alta complejidad entonces: - Calcular porcentaje_extra duplicando el número indicado a la derecha del punto en el IDC10.	

	Sino (el tratamiento NO es de alta complejidad): - El porcentaje_extra debe ser igual al 15% del monto base en todos los casos. Sea $\text{monto_final} = \text{monto_base} + \text{porcentaje_extra}$.	
3	Sea $\text{monto_extra} = 0$ Si el tratamiento es de alta complejidad, entonces: - Calcular el monto_extra como el 30% del monto base Sin importar la complejidad, adicionar al monto_extra los siguientes valores: - Si la letra ICD10 está entre "A" y "L", un monto fijo de 20.000 - Si la letra ICD10 está entre "M" y "P", un monto fijo de 15.000 + 5.000 x <Bloque ICD10> - Cualquier otra letra: 10% del monto base Si monto_extra supera a 60.000, entonces limitar el monto_extra a 60.000 Sea $\text{monto_final} = \text{monto_base} + \text{monto_extra}$	

● **Requerimiento:**

Se solicita desarrollar un programa en Python, controlado por menú de opciones, que gestione los datos de todos los tratamientos incluidos en el archivo csv provisto. Las opciones del menú se definen como sigue:

Opción 1: Cargar Tratamientos

En esta opción de menú, el programa debe procesar el archivo CSV (línea por línea) y cargar cada uno de los tratamientos a un vector de tratamientos. Al finalizar este procesamiento, el programa debe mostrar los siguientes resultados:

- **r1.1:** Cantidad de tratamientos cargados.
- **r1.2:** El apellido del paciente del **quinto** tratamiento de alta complejidad procesado. (Si no hubiera 5 tratamientos de alta complejidad, mostrar **"No hay suficientes tratamientos de alta complejidad."**)

Opción 2: Mostrar Resultados:

En esta opción de menú, el programa debe procesar el vector que se generó en la opción 1. Al finalizar este procesamiento, el programa debe mostrar los siguientes resultados:

- **r.2.1:** La diferencia promedio entre el monto final y el monto base de todos los tratamientos en el vector.
- **r.2.2:** La letra del código ICD con mayor cantidad de tratamientos registrados en el vector.
- **r.2.3:** La cantidad de tratamientos procesados en el vector para la letra obtenida del resultado **r.2.2**.
- **r.2.4:** El DNI del tratamiento de mayor monto final entre aquellos que son de alta complejidad en el vector.

- **Sobre las entradas, salidas y otros requerimientos**

El TP3 será evaluado usando un programa de control tipo runner, en forma similar a lo hecho para los TP3 anteriores. Por lo tanto, es MUY IMPORTANTE que el programa que presenten respete las exigencias siguientes:

1. El archivo csv que se provee como entrada, se llama “tratamientos.csv”. Cuando usen la función `open()` para abrir ese archivo, deben asegurarse que ese (y no otro) sea el nombre del archivo enviado a la función. Un nombre diferente, provocará un error en la ejecución del runner, y penalización de puntos en la calificación.
2. Mínimamente, el programa debe contar con dos módulos. Uno para la definición de la clase y sus métodos asociados, y otro para el programa principal (conteniendo al menú). Que se entienda: el archivo de código fuente que contiene al programa principal, DEBE llamarse `principal.py`: otro nombre, podría provocar un error en el runner.
3. Todo módulo que entreguen (que contenga un script ejecutable, incluido el módulo principal) DEBE tener control de ejecución mediante la variable `__name__`.
4. Cuando se le solicite al usuario que elija una opción de menú, la misma solicitud debe indicarse con la leyenda: “**Ingrese opción:**”. No importa la forma en la que se presenten las opciones, pero el pedido de opción debe respetar ese mensaje en la consola. Y por supuesto, la carga del número de opción debe hacerse desde el teclado (siendo estas las únicas cargas por teclado permitidas en el programa).
5. La opción para salir debe ser la opción 0. Nuevamente, no importa el nombre con el que se muestre esta opción en el menú, cuando el usuario ingrese la opción 0, el programa debe terminar. No muestre un mensaje de cierre. No muestre un mensaje de despedida. No muestre íconos tristes. Ni felices. Nada. Solo termine.
6. Los resultados deben en mostrarse en el orden solicitado, y utilizando los dos puntos (":") para separar el texto aclaratorio del resultado en sí. Si, por ejemplo, en la opción 1 se piden los resultados `r1.1` y `r1.2` y el programa calcula `r1.1 = 5475.34` y `r1.2 = 3`, entonces la salida (Cuando se seleccione la opción 1) debe ser:

```
Ingrese opción: 1
r1.1: 5475.34
r1.2: 3
```

7. NO PUEDEN usar en este TP3 estructuras de datos que no hayan sido tratadas en clase o en las fichas que abarca este trabajo. No usen diccionarios. No usen conjuntos. Usen una lista/arreglo de registros/objetos. Si tienen que hacer un ordenamiento, deben programar ustedes ese ordenamiento (no usar una función nativa que lo haga). Si tienen que hacer una búsqueda en el vector de registros, deben programar ustedes esa búsqueda.
8. DEBEN usar funciones correctamente planteadas en cada lugar donde sea oportuno y aplicable. No es opcional. **ES OBLIGATORIO USAR FUNCIONES EN TODA SITUACIÓN POSIBLE**. No dispongan de un script con instrucciones sueltas. El arranque del programa DEBE estar contenido en una función principal, que luego invoque a otras funciones.